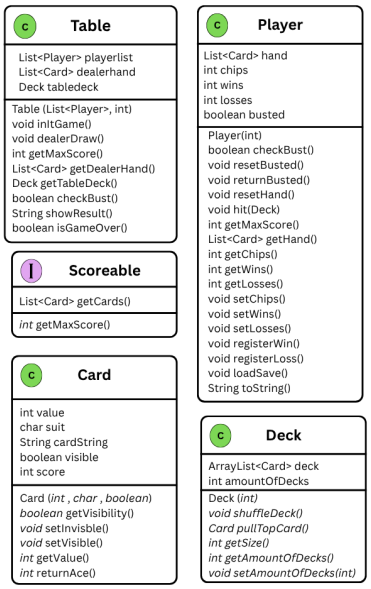
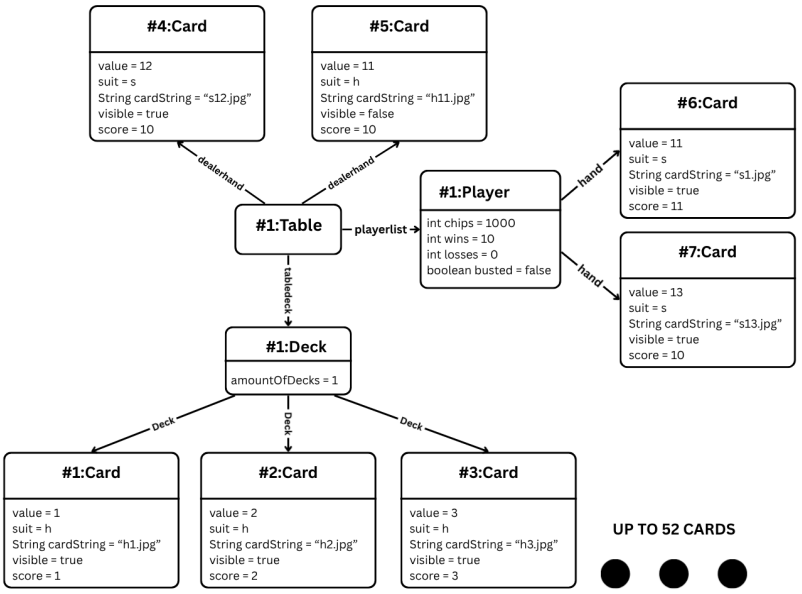
Beskrivelse av appen

Vi har laget en enkel implementasjon av kortspillet Blackjack i Java og JavaFX. Spillet består av en spiller som konkurrerer mot en “Dealer”. Målet er å samle “chips”, som er tilsvarende penger i dette spillet. Blackjack er et rimelig enkelt kortspill hvor målet er å få en hånd som er nærmest mulig, men ikke over, 21. Både spiller og Dealer starter med to kort, men vi får bare se det ene kortet til Dealer. Spilleren kan da velge å trekke flere kort (“hit”) eller si seg ferdig (“stand”). Bildekort teller alle som 10 og ess kan være enten 11 eller 1.
Appen tillater også spilleren å velge hvilke design på kortstokken som skal tas i bruk. Spillet mangler noen av vanlige funksjonene i Blackjack, blant annet muligheten for split og mer enn 1 spiller, men den grunnleggende gangen i spillet var fokuset vårt.
Vi implementerte også en save og load funksjon som teller antall wins og losses og antall chips som brukeren har igjen og kan lades gjennom å trykke på knappen i menuen.

Diagram





KLASSEDIAGRAM | OBJEKTDIAGRAM

Spørsmål

Hvilke deler av pensum dekkes av prosjektet vårt?

Vi har brukt de grunnleggende delene i pensum. Vi har brukt:

- klasser og instanser, felt, metoder og konstruktører, bruk av this
- innkapsling, interaksjon mellom objekter, standardmetoder (navngiving og funksjon) ved innkapsling
- objektoppførsel, grensesnitt og implementasjon av grensesnitt
- diagrammer:
    - objektdiagrammer (objekter med tilstand og koblinger)
    - klassediagrammer (klasser og assosiasjoner inkl. kardinalitet)
- testing av objektoppførsel (JUnit og generell testbeskrivelse lik det gjort på forelesning og i øvinger)
- unntakshåndtering

Hvilke deler av pensum dekkes ikke?

Vi har ikke brukt:

- diagrammer:
    - sekvensdiagrammer
    - objekttilstandsdiagrammer
- arv (bruk av extends og super) og abstrakte klasser (også i kombinasjon med grensesnitt)

Hvordan forholder koden vår seg til Model-View-Controller-konseptet?

Koden vår er skrevet med hensyn til vår oppfatning av MVC-konseptet. Implementasjonen av Blackjack-funksjon og spillets gang kontrolleres av Java-koden. Denne håndterer kortstokken, oversikt over kort på hånd, og resultatberegning osv. Dette er altså “Model” i vår app. “Controller” i vårt tilfelle er AppController.java som tar inn input fra brukeren og bruker kall til Blackjack-funksjonaliteten for å finne hva som skal skje videre, for å deretter be JavaFX om å vise dette. Vi valgte å lage en liste med imageviews og brukte dem som “placeholders” der alle er satt til setVisible(false), der vi i AppController har laget en metode som går gjennom spillerens og dealerns/tables sine hender (alle kortene de har på hånd) og kobler da hvert kort i hånden til en tilsvarende ImageView ved å laste inn riktig kortbilde basert på kortets verdi og farge. Dersom kortet er synlig, vises kortets faktiske bilde, mens usynlige kort viser baksiden. “View” i vår app er definert i App.fxml-filen, som beskriver brukergrensesnittet med knapper, labels og “placeholders”. JavaFX renderer dette og AppController oppdaterer det via FXML.

En (vennlig) diskusjon oppsto mellom oss rundt hvilke deler av kontrollen som tas av “Controller” og “Model”. Etter som vi hadde fordelt ansvaret for utviklingen mellom oss i starten hadde nok begge sine egne ideer om hva som var ansvaret til de ulike delene av appen. Jeg som utviklet Blackjack-funksjonalitet til å begynne med hadde et syn på at “Model” mer eller mindre hadde hele spillet gående og kun spurte Controller og View om

input når den sa at den trengte det. Eksempelvis så var tanken at kort ble delt ut og Model deretter sa ifra om at “Hei, nå venter jeg på at noen sier Hit eller Stand” og “Her er kortene som er på hånd”. Dette viste seg i implementasjonen å bli kronglete og litt tungvint, så vi endte etterhvert med å overføre flere deler av koordinasjonen over til AppController. Dette ga oss en ryddigere kontroll over spillets gang og interaksjonene med GUIen.

Hvordan tok vi i bruk tester i utviklingen av koden vår?
Utviklingen av appen foregikk hovedsaklig i tre stadier: Utvikling av Blackjack-funksjonalitet, implementasjon av JavaFX-grensesnitt og koblingen av disse. Hver av disse stadiene hadde sine egne behov for testing.

For testing av Blackjack-funksjonalitet brukte vi en Main-metode som spilte spillet gjentatte ganger med tilfeldig stokket kortstokk hver gang. Et viktig punkt her er at kortstokken var unik for hver runde, noe som gjorde det mye lettere å oppdage grensetilfeller og uønsket oppførsel. For hver runde i Main-testen brukte vi en toString-metode som skrev ut kort på hånd og kalkulert resultat. Slik kunne vi gå gjennom og sjekke at ting fungerte slik de skulle på et stort antall unike instanser. Ikke alle deler av koden ble testet i isolasjon, men etter som mye av koden kun ble tatt i bruk av andre metoder med klare bruksområder og parametere så var det ikke nødvendig etter som testen på det sammensatte resultatet fungerte på alle grensetilfeller.

Etterhvert laget vi en JUnit-test (UnitTest.java) for å teste visse grensetilfeller og oppførsel. Dette skulle vi nok gjort tidligere, men main-testen fungerte ganske fint for preliminære tester. Med JUnit-testene fikk vi mer nøyansert testing av metodene til klassene, imens main-testen stort sett testet hvorvidt spillets gang gikk som den skulle.

KI-deklarasjon

Tro det eller ei, men vi har anvendt KI i dette prosjektet. Vi har begge brukt Claude kjørende i terminalen begge to. Vi har hatt løpende diskusjon om hvordan vi bruker KI for å oppnå våre mål. En sentral ide vi har hatt er at vi ønsker å stille å snevre spørsmål som overhode mulig til KI-en; spisse spørsmålet ned til akkurat det du lurer på for å unngå å få hele svaret i fanget. Magnus fra Algdat sa at “Confusion is the sweat of learning”, noe som jeg tror veldig på. Når man har en eller annen problemstilling i data-verden så ligger det ofte mye læring mellom deg og løsningen på svaret. Tre timer på StackOverflow og 6-7 år gamle Reddit-tråder er en kjent situasjon for mange som har tuklet litt med datamaskiner. En stor

del av læringen i dette prosjektet går ut på å gjøre oss såpass forvirret at vi rett og slett er nødt til å lære oss Java for å få nok forståelse til å gjøre det vi vil. KI, derimot, har en ganske enkel sti: Problemstilling -> Løsning. Derfor unngår man den læringen og dybdeforståelsen man får av å knote med problemet selv.

Derimot så ville jeg sagt at det er trangsynt og umoderne å tenke at den beste innfallsvinkelen alltid er å gjøre ting selv uten KI-hjelp. En hyppig mening som ytres er at "folk er dårligere til å kode pga KI". En annen innfallsvinkel på dette kan være at "folk får til mere med mindre kodekunnskaper". Målet med dette prosjektet er å gi oss et visst grunnlag i Java og OOP generelt. Målet vårt med KI burde da være å anvende denne teknologien til å gjøre mere med det grunnlaget vi danner oss i faget og prosjektet. Rettere sagt, KI burde la oss gjøre mer med det grunnlaget vi har, ikke la oss nå minimumskravet med dårligere grunnlag.

Men hvordan ser dette ut i praksis i vår KI-bruk? KI funker veldig bra til å rydde opp i eget rot, og eventuelt få tilbakemelding på hvorvidt måten du har skrevet kode er i tråd med kotymer og regler du ikke vet om.
Si at man skal implementere en slags datastruktur. Først ville jeg prøvd selv til jeg har noe som fungerer eller møter på noe jeg ikke kan. Hvis jeg har noe som fungerer kan jeg spørre "Er dette en trygg/effektiv/gunstig måte å implemetere denne datastrukturen?" Da kan KI-en si f.eks "Tja, nesten. Her er det nok raskere å bruke en hashtabell, men den øvrige implementasjonen er bra utenom (...)". Dette gir meg innsikt som jeg ellers ikke er i stand til å se selv og åpner muligheten for at jeg kan innse hva det er jeg ikke forstår enda. Ellers kunne det hendt at jeg fortsatte å danne dårlig vane og forståelse.

For å svare på spørmålene mer konkret:

- Nei, vi trengte ikke KI for løse oppgaven
- Vi brukte begge utelukkende Claude

Claude skrev ytterst lite av den koden som ble en del av prosjektet, men har stort sett vært veiledende og oppklarende. Eksempelvis brukte vi Claude til å oppdage snåle logikkfeil i koden som ikke var lette å nøste opp i, eller til å forklare konsepter og funksjoner i Java/JavaFX iht. vår oppgave.